Claude Code in der Praxis

Von der Idee zum KI-gestützten Entwicklungs-Workflow

Interne Weiterbildung · Hands-on Workshop

Agenda

Block 0	**Setup & App kennenlernen**
Block 1	**Projekt initialisieren (/init, CLAUDE.md, Tests)**
Block 2	**Subagents einrichten**
Block 3	**Skills erstellen**
Block 4	**Hooks konfigurieren**
Block 5	**Wrap-up & Ausblick**

Pro Block: Konzept → Aufbau → Hands-on

BLOCK 0

Setup & App kennenlernen

Voraussetzungen

- IntelliJ IDEA installiert
- Java 21 + Maven
- Aktives Claude-Abo (Pro/Team)
- Node.js (für Claude Code CLI)

Schritte

1. CLI: npm install -g @anthropic-ai/claude-code
2. IntelliJ Plugin installieren
3. Repository klonen
4. mvn spring-boot:run → localhost:8085
5. claude starten (Terminal/Plugin)

```
$ claude
Claude Code  ·  Bereit für Eingaben...
```

BLOCK 1

Projekt initialisieren

CLAUDE.md

Das technische Briefing

- Build- & Run-Commands
- 3-Layer-Architektur
- Testing-Regeln
- Konfiguration (Port, DB)

requirements.md

Nur fachliche Anforderungen

- Entity-Felder & Validierung
- CRUD-Operationen & URLs
- Geschäftsregeln
- Startdaten

Keine technischen Details in requirements.md — diese gehören in CLAUDE.md. Halte CLAUDE.md kurz und prägnant.

Neu: Testing-Sektion in CLAUDE.md — nach Änderungen mvn test, fehlende Tests anbieten (Mockito)

HANDS-ON

CLAUDE.md, requirements.md & Tests

1. **/init eingeben — Claude analysiert das Projekt**
   Automatische Erkennung von Stack, Struktur, Commands

2. **CLAUDE.md prüfen und ergänzen**
   3-Layer-Architektur + Testing-Sektion (Mockito, Repository mocken)

3. **requirements.md manuell erstellen**
   Nur fachliche Anforderungen: Felder, Validierung, URLs, Geschäftsregeln

4. **Unit-Tests für PersonService generieren**
   Claude bitten, Tests zu schreiben — verifizieren mit mvn test

Musterlösung: `git checkout reference/init`

BLOCK 2

Subagents

Spezialisierte KI-Agenten für wiederkehrende Aufgaben

Eigenschaften

- Eigener Name & Beschreibung
- Eingeschränkte Tools (z.B. nur Lesen)
- Eigenes Modell (sonnet / opus)
- Definiertes Ausgabeformat

Einsatzgebiete

Code-Review	Architektur-Compliance prüfen
Erklärungen	Code auf bestimmtem Niveau erklären
Testing	Automatisch Tests generieren
Dokumentation	Code dokumentieren & beschreiben

Unsere Workshop-Agents

`code-erklaerer-efz`

Deutsch · Read-only · sonnet

`spring-mvc-code-reviewer`

English · Read-only · sonnet

Aufbau eines Subagents

```
---
name: spring-mvc-code-reviewer
description: >
  Reviews code for architecture
  compliance. Read-only.
tools: Read, Grep, Glob
model: sonnet
color: blue
---

You are an experienced Java...

## Mandatory first steps
## What to review
## Output format
```

Frontmatter (YAML)

Sprache: immer Englisch
Bestimmt wann Claude
den Agent automatisch startet

System-Prompt (Markdown)

Sprache = Ausgabesprache
Rolle, Pflichtschritte,
Ausgabeformat, Grenzen

Read-only Agents (tools: Read, Grep, Glob) sind sicher und können jederzeit eingesetzt werden

HANDS-ON

Subagents erstellen & testen

1. **Ordner .claude/agents/ erstellen**
   mkdir -p .claude/agents

2. **Frontmatter definieren**
   name, description, tools: Read/Grep/Glob, model: sonnet

3. **System-Prompt schreiben**
   Rolle, Pflichtschritte (CLAUDE.md lesen!), Ausgabeformat

4. **Testen: Claude bitten, Code zu reviewen**
   „Review den PersonController auf Layer-Violations"

Bonus: Zweiten Agent code-erklaerer-efz.md erstellen — Deutsch, für Lernende im 2. Lehrjahr

BLOCK 3

Skills

Wiederverwendbare Rezepte für wiederkehrende Aufgaben

	Agent	Skill
Zweck	Eigenständige Rolle	Schritt-für-Schritt Rezept
Ort	.claude/agents/	.claude/skills/<n>/
Eigenes Modell?	Ja (model: sonnet)	Nein
Darf schreiben?	Nein (read-only)	Ja (Write, Edit)
Typisch für	Review, Erklärung	Scaffolding, Migration

Der wichtigste Step im Skill: Bestehenden Code lesen, bevor neuer Code generiert wird!

Beispiel: entity-scaffold

Generiert eine komplette CRUD-Entity: Model → Repo → Service → Controller → Templates → DataInit → Nav

Entity-Scaffold: 10 Schritte

0	1	2	3	4
Verstehen	**Code lesen**	**Entity**	**Repository**	**Service**
Klärungsfragen	Stil lernen	JPA + Lombok	Spring Data	@Transactional

5	6	7	8	9
Controller	**Templates**	**DataInit**	**Navigation**	**Verify**
URL-Schema	list, form, detail	Beispieldaten	layout.html	mvn compile

Step 1 ist entscheidend — Claude lernt den bestehenden Code-Stil, bevor neuer Code generiert wird

HANDS-ON

Entity-Scaffold Skill erstellen & anwenden

1. **.claude/skills/entity-scaffold/SKILL.md erstellen**
   Frontmatter + 10 Steps mit Code-Beispielen

2. **Wichtig: Step 1 = Bestehenden Code lesen!**
   Damit generierter Code zum Stil des Projekts passt

3. **Testen: "Erstelle eine neue Entity Address"**
   street, houseNumber, postalCode, city, country — @ManyToOne zu Person

`Musterlösung: git checkout reference/skills`

Die Musterlösung enthält den Skill und die damit generierte Address-Entity inkl. aller Schichten + Tests

BLOCK 4

Hooks

Shell-Skripte, die automatisch vor oder nach Claude-Aktionen laufen

Hook-Typen

`PreToolUse`	Vor Tool-Nutzung → Dateien schützen
`PostToolUse`	Nach Tool-Nutzung → Linting, Format
`Notification`	Bei Events → Benachrichtigungen

Exit-Codes

`exit 0`	Aktion erlauben
`exit 1`	Hook-Fehler (erlaubt trotzdem)
`exit 2`	Aktion blockieren

Beispiel: Config-Schutz

Claude will application.properties ändern → PreToolUse Hook wird ausgelöst → protect-config.sh prüft Dateiname → exit 2 BLOCKIERT

HANDS-ON

Config-Schutz-Hook implementieren

1. **Claude bitten, den Hook zu schreiben**
   protect-config.sh: Dateipfad auslesen, Dateiname prüfen, exit 0 oder 2

2. **Hook in .claude/settings.json registrieren**
   PreToolUse, matcher "Write|Edit", timeout 5s

3. **Testen: Claude bitten, Port zu ändern**
   Claude sollte blockiert werden und die Fehlermeldung sehen

`Musterlösung: git checkout reference/hooks`

Weitere Hook-Ideen

`PostToolUse`	Linting nach Änderungen (mvn checkstyle:check)
`PreToolUse`	Test-Files vor Löschung schützen
`PostToolUse`	Auto-Format nach jedem Write

Alternativen & Übertragbarkeit

Konzept	Claude Code	GitHub Copilot	OpenAI Codex
Projekt-Kontext	CLAUDE.md	copilot-instructions.md	AGENTS.md
Agenten	.claude/agents/	.github/agents/ *.agent.md	Codex Subagents
Skills	.claude/skills/	Skills (awesome-copilot)	.codex/skills/ (SKILL.md)
Hooks	.claude/hooks/	Agent Hooks (Preview)	userpromptsubmit Hook

AGENTS.md ist ein offener Standard unter der Linux Foundation, unterstützt von Codex, Copilot, Jules, Cursor u.a. Claude Code liest ebenfalls AGENTS.md-Dateien. Die Grundidee ist überall gleich: Je besser du dem KI-Tool dein Projekt erklärst, desto besser das Ergebnis.

Was wir heute gelernt haben

CLAUDE.md + requirements.md	Technisches Briefing + fachliche Spezifikation
Subagents	Spezialisierte, sichere KI-Rollen (read-only)
Skills	Wiederverwendbare Rezepte (entity-scaffold)
Hooks	Automatischer Schutz sensibler Dateien

Dein .claude/ Ordner

```
.claude/  agents/ · hooks/ · skills/ ·
settings.json
```

Weiterführend

code.claude.com/docs
agents.md · developers.openai.com/codex/skills

Viel Spass beim Experimentieren!

Fragen? Feedback? Ideen?